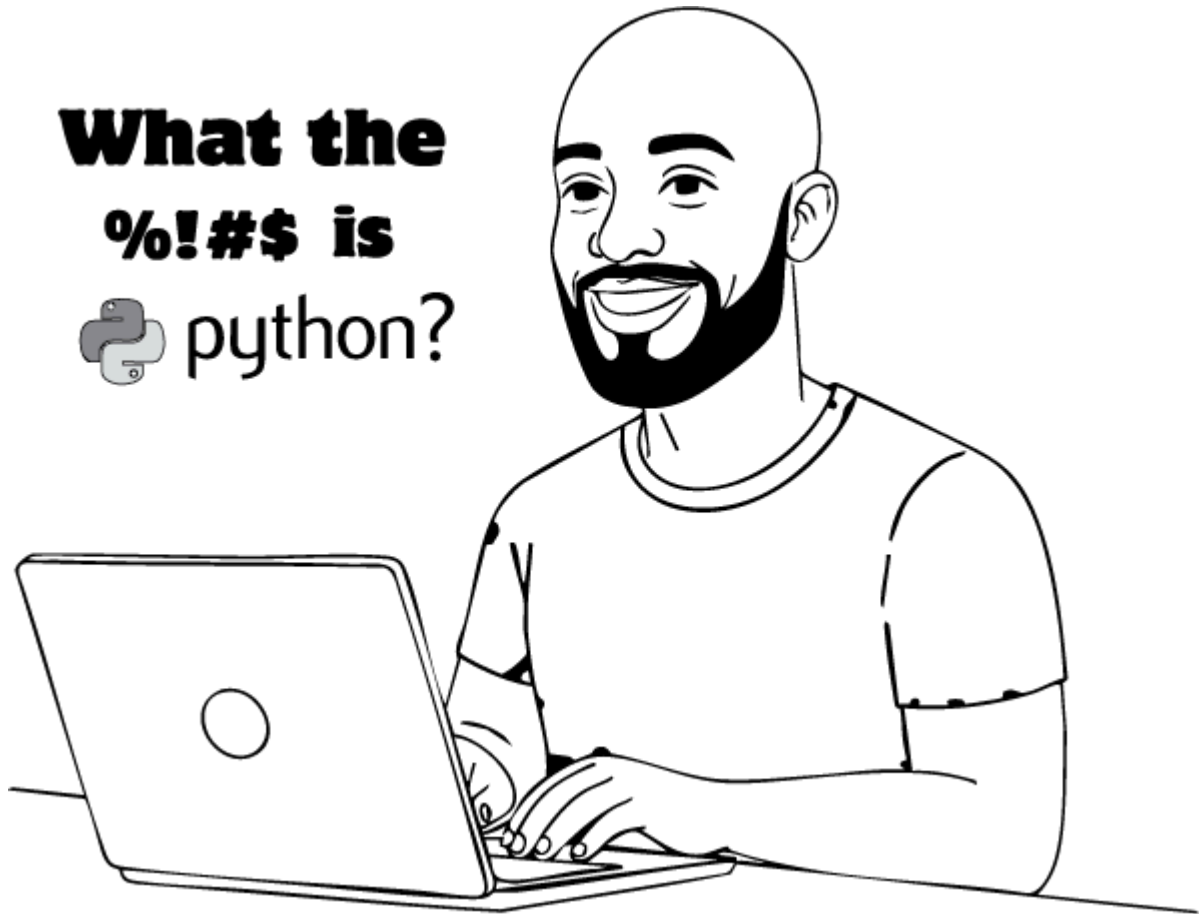


**What the
%!#\$ is
 python?**



A BEGINNERS GUIDE TO SOLVING PROBLEMS USING CODE

WHO IS PYTHON FOR?

Python is for anyone who wants to solve problems with code — from total beginners to seasoned professionals. Its users include students, educators, technical writers, data analysts, scientists, web developers, and hobbyists alike. In short, Python is for everyone... well, except actual snakes.

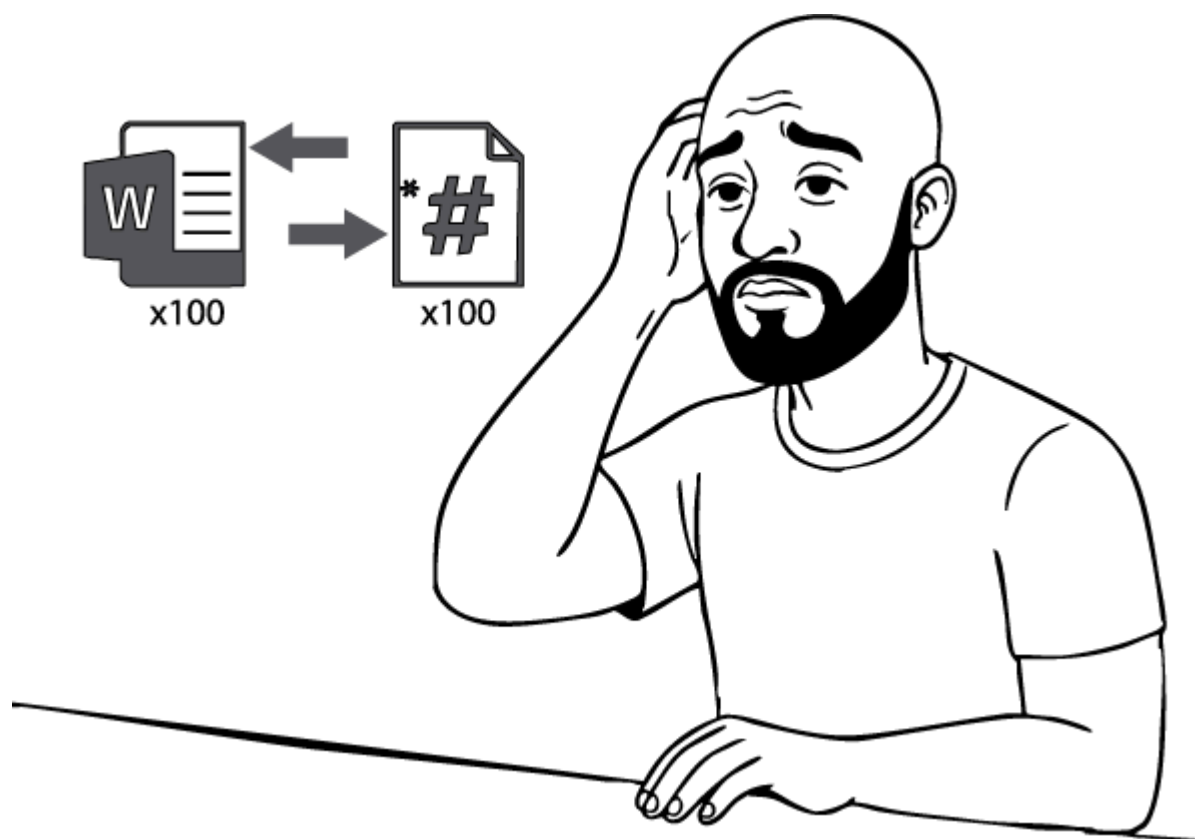


I Prefer
SsssQL anyway!

WHY DO TECHNICAL WRITERS USE PYTHON?

Python is becoming increasingly valuable in **technical writing**, especially as the role evolves to include more automation, toolchain integration, and data-driven documentation. Tech writers use Python to:

- Automate repetitive tasks like bulk converting files (i.e. Word files to Markdown files), renaming and organizing documentation files, searching for and replacing content across multiple files
- Generate Markdown or HTML pages automatically from templates and metadata
- Ensure XML content conforms to a schema (such as DITA) prior to publication
- Format or lint Markdown and other plain text formats
- Creating internal tools that generate boilerplate docs or dashboards for doc versioning or status checks
- Making test API calls and documenting real responses
- And much more...



MODULE 1: INSTALL VS CODE AND PYTHON EXTENSION

The goal of this module is to get your computer ready to write and run Python code.

Setup

1. Install Python

1. Download from python.org
2. Check "Add Python to PATH" during installation.

2. Install Visual Studio Code (VS Code)

1. Download from <https://code.visualstudio.com/>

3. Install the Python Extension in VS Code

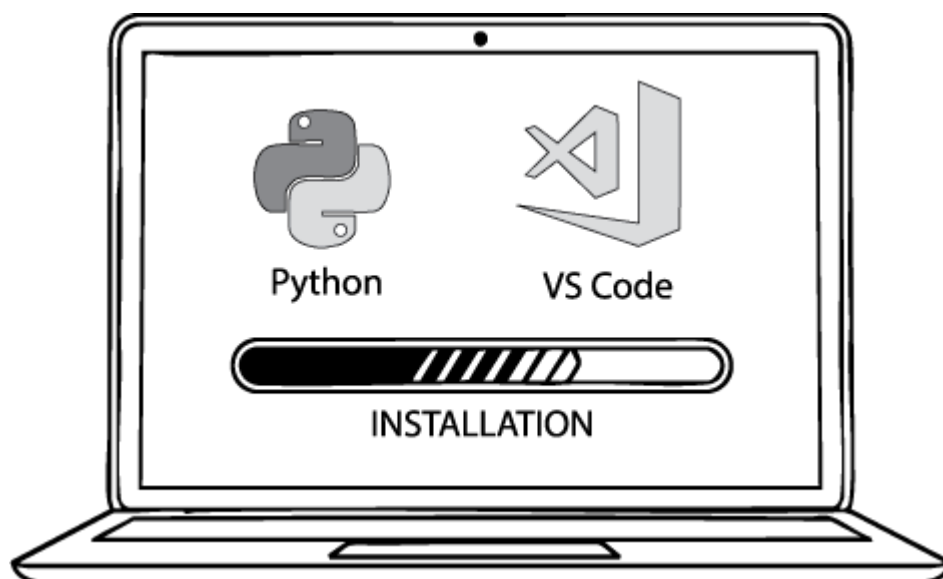
1. Open VS Code → Extensions tab (or `Ctrl+Shift+X`) → Search "Python" → Click **Install**

4. Verify Python Is Installed

1. Open Terminal in VS Code (`Ctrl + backtick`)
2. Type: `python --version` or `python3 --version`

Note

To open the terminal in VS code, type (`Ctrl + backtick`) or go to the top menu → **View** → **Terminal**.



MODULE 2: CONFIGURE PYTHON INTERPRETER

The "interpreter" is the actual Python program that runs your code. You might have more than one Python version installed, so you need to tell VS Code which one to use.

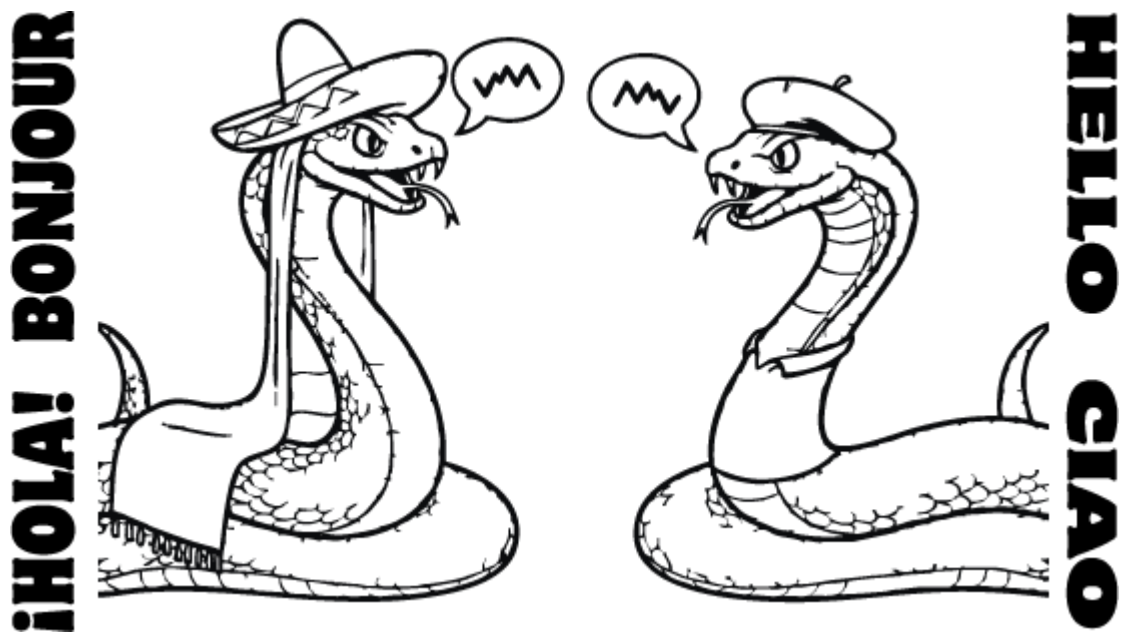
Setup

1. Configure Python Interpreter

1. Open the Command Palette (`Ctrl+Shift+P`)
2. Type: `Python: Select Interpreter`
3. Select the following interpreter: `Python 3.x (C:\Users\YourName\AppData\...)`

Note

If you don't see any options, restart VS Code and try again.



MODULE 3: CREATE AND RUN SCRIPT

Create a New Script

1. In VS Code, click **File > Open Folder**, and create/select a folder like `python-course`
2. In the **Explorer** tab (left sidebar), click **New File** and name it: `hello.py`
3. In the file, type: `print("Hello, technical writer!")`

Run the Script

- Click the green **"Run Python File"** button in the top-right corner, **OR** open the terminal (`View > Terminal` or `Ctrl + ```)
- Type: `python hello.py`



KEY AREAS IN VS CODE

PANEL	PURPOSE
Explorer	Browse your project files and folders
Editor	Write and view your Python code
Terminal	Run commands like "python myfile.py"
Extensions	Add new features like Python, Markdown tools, etc.
Source Control	Use Git for version control (optional for now)

BASIC SHORTCUTS

KEYBOARD KEYS	ACTION
Ctrl+P	Quick open a file
Ctrl+Shift+P	Command Palette (do anything)
Ctrl+` (backtick)	Open terminal
Ctrl+S	Save your file
Shift+Alt+F	Format code neatly (after installing a formatter)

MODULE 4: PRACTICE PYTHON FUNDAMENTALS

Practice using these Python building blocks to start writing readable, structured scripts.

Create a New Python File

1. In VS Code, click File > Open Folder, and create/select a folder like python-course
2. In the Explorer tab (left sidebar), click New File and name it: **basics.py**

Practice fundamental tasks

Type the following text into the terminal and observe the outputs.

DEFINITIONS	CODE
A VARIABLE stores information (like a name or number) so you can reuse it later.	<pre>name = "Sam" # a string (text) age = 34 # an integer (number) is_writer = True # a boolean (True or False)</pre>
Use PRINT() to show the result of your code in the terminal.	<pre>print("Name:", name) print("Writer?", is_writer)</pre> The output will look like this: Name: Sam Writer? True
Use + for basic combinations of text and variables. (Should be used rarely)	<pre>print(name + " is a technical writer.")</pre>
Use F-STRING for all other combinations of text and variables. (Best practice)	<pre>print(f"{name} is a technical writer aged {age}.")</pre>
Include MATH SYMBOLS for basic equations. + addition - subtraction * multiplication / division	<pre>docs_written = 10 days = 5 print("Docs per day:", docs_written / days)</pre> The output will look like this: Docs per day: 2.0
A COMMENT starts with # and is ignored by Python. It's just for humans reading the code.	<pre># This script prints a summary for a technical writer</pre>

MODULE 5: WORKING WITH FILES AND TEXT

Practice using Python scripts to read and write files like .txt and .md.

Create a New Python File

1. In VS Code, click File > Open Folder, and create/select a folder like python-course
2. In the Explorer tab (left sidebar), click New File and name it: **file_editor.py**

Practice the following automations

Type the following text into the terminal and observe the automation.

AUTOMATION	CODE
Read a Markdown file	<pre>with open("example.md", "r", encoding="utf-8") as f: content = f.read() print(content)</pre>
Write to a new file	<pre>with open("summary.md", "w", encoding="utf-8") as f: f.write("# Documentation Summary\n\nWritten with Python!")</pre>
Search and replace in file content	<pre>content = content.replace("outdated_term", "updated_term")</pre>
Loop through multiple files	<pre>import os for filename in os.listdir("docs"): if filename.endswith(".md"): print("Found file:", filename)</pre>

MODULE 6: AUTOMATING DOCUMENTATION TASKS

Use Python to simplify and speed up repetitive doc tasks.

Create a New Python File

1. In VS Code, click File > Open Folder, and create/select a folder like python-course
2. In the Explorer tab (left sidebar), click New File and name it: **doc_automation.py**

Practice the following automations

Type the following text into the terminal and observe the automation.

AUTOMATION	CODE
Generate a changelog from a list	<pre>changes = ["Fixed typos", "Updated API section", "Added new diagrams"] with open("CHANGELOG.md", "w") as f: for change in changes: f.write(f"- {change}\n")</pre>
Convert <code>.txt</code> to <code>.md</code>	<pre>with open("draft.txt") as fin: text = fin.read() with open("draft.md", "w") as fout: fout.write("# Converted File\n\n" + text)</pre>
Pull data from a mock API (bonus)	<pre>import requests response = requests.get("https://jsonplaceholder.typicode.com/posts/1") data = response.json() print(data["title"])</pre>
Auto-generate Markdown documentation	<pre>doc = f"# {data['title']}\n\n{data['body']}" with open("api_doc.md", "w") as f: f.write(doc)</pre>

MODULE 7: CHECK ON KNOWLEDGE

Perform the following challenge.

- Create a CLI tool that:
 - Reads a folder of `.md` files
 - Generates a report of word counts, titles, and missing metadata
 - Outputs the summary to a `docs_report.md`